

Introduction

Picture your calendar app. You've got a meeting from 2:00 to 3:00. Someone tries to book 2:45 to 3:30. The app has to flag that conflict instantly, every time, with zero ambiguity. That's the real-world shape of LeetCode 836: Rectangle Overlap.

It's labeled "easy," and the core idea genuinely is easy once you see it. But the problem hides a few sharp edges — the kind that trip up developers who've solved plenty of harder problems but jump straight to geometry instead of stepping back and reframing the question. Interviewers like this problem precisely because it rewards a specific instinct: turning "when do these things overlap" into "when do these things definitely miss." That reframe — checking the complement of what you actually want — shows up constantly in interval scheduling, memory allocation, calendar systems, and range-based systems design. Rectangle Overlap is a clean, low-stakes place to build that instinct before you need it somewhere expensive.

Problem Overview

You're given two rectangles on a 2D grid. Each one is described by its bottom-left corner and its top-right corner — four numbers total per rectangle. You need to return `true` if the rectangles overlap with a genuine, positive area, and `false` otherwise.

The critical rule: touching isn't overlapping. If two rectangles share only an edge or a single corner, that's zero area of intersection, and the answer is `false`. You're also guaranteed that every rectangle is valid — its bottom-left corner is strictly below and to the left of its top-right corner, so you never have to defend against a degenerate, zero-area rectangle as input.

Example

Let's set up two rectangles:

- `rec1 = [0, 0, 2, 2]` — bottom-left at (0,0), top-right at (2,2)
- `rec2 = [1, 1, 3, 3]` — bottom-left at (1,1), top-right at (3,3)

Rec1 spans x from 0 to 2 and y from 0 to 2. Rec2 spans x from 1 to 3 and y from 1 to 3. Visually, rec2 is shifted up and to the right, but it still cuts through rec1's upper-right quadrant. The overlapping region is the square from (1,1) to (2,2) — real area, so the answer is `true`.

Now compare that to `rec1 = [0, 0, 2, 2]` and `rec2 = [2, 0, 4, 2]`. Rec2 starts exactly where rec1 ends on the x-axis. They share the vertical edge at $x=2$, but there's zero width of overlap. The answer here is `false`, even though the rectangles are touching.

Intuition

The natural first instinct is to check whether a corner of one rectangle lands inside the other. It feels obvious — if they overlap, surely one rectangle pokes a corner into the other, right?

Wrong, and this is exactly the trap the problem sets. Picture a long, thin rectangle sliced straight through the middle of a wide one, like a knife through a stick of butter. The thin rectangle overlaps the wide one significantly — real area, unmistakably — but none of its four corners land inside the wide rectangle, and none of the wide rectangle's corners land inside the thin one. Corner-checking misses this case entirely.

So flip the question. Instead of asking "do these overlap," ask "when could they definitely *not* overlap?" There are exactly four ways two rectangles can fail to touch:

1. Rec1 is entirely to the left of rec2
2. Rec1 is entirely to the right of rec2
3. Rec1 is entirely above rec2
4. Rec1 is entirely below rec2

If none of those four conditions hold, the rectangles must overlap — no corner-checking, no geometry, no edge cases about diagonal slices. This is the same move you'd make reasoning about two meetings on a calendar: instead of asking "do these time ranges overlap," you ask "does one end before the other starts." Reduce the rectangle problem to that same one-dimensional question, once for x and once for y , and require both to hold.

Brute Force Solution

Idea: Check every corner of rec1 against rec2's boundaries, and every corner of rec2 against rec1's boundaries, looking for any point that falls strictly inside the other rectangle.

Algorithm: For each of the four corners of each rectangle, test whether that point lies strictly within the other rectangle's bounds.

Advantages: Feels intuitive, matches how people first picture the problem visually.

Disadvantages: It's wrong. As shown above, a thin rectangle can overlap another with zero corners inside either shape. This approach fails silently on that exact case, which is worse than being slow — it's incorrect.

Time complexity: $O(1)$ — still just a constant number of checks. **Space complexity:** $O(1)$.

Since this approach doesn't actually solve the problem correctly, it's not a real fallback — it's a trap to recognize and route around, which is why the reframe below is the only solution worth writing.

Optimal Solution

Project both rectangles onto the x-axis, ignoring y entirely. Rec1 becomes the interval `[x1, x2]`, rec2 becomes `[x3, x4]`. Do the same on the y-axis: `[y1, y2]` and `[y3, y4]`.

Two intervals overlap with positive length exactly when each one's start is strictly before the other's end:

```
x_overlap = rec1.x1 < rec2.x2 and rec2.x1 < rec1.x2
y_overlap = rec1.y1 < rec2.y2 and rec2.y1 < rec1.y2
```

Check

Meaning

<code>rec1.x1 < rec2.x2</code>	rec1 doesn't start after rec2 ends (x-axis)
<code>rec2.x1 < rec1.x2</code>	rec2 doesn't start after rec1 ends (x-axis)
<code>rec1.y1 < rec2.y2</code>	rec1 doesn't start after rec2 ends (y-axis)
<code>rec2.y1 < rec1.y2</code>	rec2 doesn't start after rec1 ends (y-axis)

The rectangles overlap with real area only when **both** the x-projection and y-projection overlap. The strict `<` is what excludes edge-touching and corner-touching — a shared boundary produces equal values on one side, which fails the strict inequality and correctly returns `false`.

Python Code

```
class Solution:
    def isRectangleOverlap(self, rec1: list[int], rec2: list[int]) -> bool:
        x1, y1, x2, y2 = rec1
        x3, y3, x4, y4 = rec2

        x_overlap = x1 < x4 and x3 < x2
        y_overlap = y1 < y4 and y3 < y2

        return x_overlap and y_overlap
```

Code Walkthrough

- `x1, y1, x2, y2 = rec1` unpacks `rec1`'s bottom-left `(x1, y1)` and top-right `(x2, y2)`.
- `x3, y3, x4, y4 = rec2` does the same for `rec2`.
- `x_overlap = x1 < x4 and x3 < x2` checks the x-axis projections: `rec1` must start before `rec2` ends, and `rec2` must start before `rec1` ends.
- `y_overlap = y1 < y4 and y3 < y2` runs the identical logic on the y-axis.
- `return x_overlap and y_overlap` — both axes must independently overlap for the rectangles to share real area.

Dry Run

Using `rec1 = [0, 0, 2, 2]` and `rec2 = [1, 1, 3, 3]`:

- `x1, y1, x2, y2 = 0, 0, 2, 2`
- `x3, y3, x4, y4 = 1, 1, 3, 3`
- `x_overlap = (0 < 3) and (1 < 2) → True and True → True`
- `y_overlap = (0 < 3) and (1 < 2) → True and True → True`
- Return `True and True → True`

Now the edge-touching case, `rec1 = [0, 0, 2, 2]` and `rec2 = [2, 0, 4, 2]`:

- `x1, y1, x2, y2 = 0, 0, 2, 2`
- `x3, y3, x4, y4 = 2, 0, 4, 2`
- `x_overlap = (0 < 4) and (2 < 2) → True and False → False`
- Return `False` immediately (short-circuits) — `False`, correctly rejecting the shared edge.

Complexity Analysis

Time: $O(1)$. Four comparisons, regardless of coordinate magnitude — there's no loop, no scan, nothing that grows with input size.

Space: $O(1)$. A handful of unpacked integers, no auxiliary structures.

These are exact bounds, not amortized estimates — the algorithm does the same fixed work for coordinates near zero or near the integer limits.

Alternative Solutions

You could compute the intersection rectangle explicitly — `max(x1, x3)` to `min(x2, x4)` for width, same idea for height — and check both resulting dimensions are positive. This is mathematically equivalent to the projection check and also $O(1)$. It's arguably more useful if you later need the actual overlapping area or region, not just a boolean, since it hands you the intersection's coordinates directly. For this problem, where only `true / false` is required, the direct interval check is more minimal — no reason to compute values you'll never inspect.

Edge Cases

- **Exact edge touch:** rectangles share a full edge (e.g., `rec2` starts where `rec1` ends on `x`). Strict `<` correctly returns `false`.
- **Corner touch only:** rectangles meet at a single point. Both axis checks fail on at least one side, returning `false`.
- **One rectangle fully inside another:** all four inequalities hold comfortably, returns `true`.
- **Identical rectangles:** every inequality holds strictly, returns `true` — they overlap with their entire area.
- **Large coordinates:** since the algorithm is pure comparison, coordinate magnitude (even near the problem's stated bounds) has zero effect on runtime.

Common Mistakes

1. **Using `<=` instead of `<`.** This incorrectly counts edge-touching or corner-touching as overlap.
2. **Checking corners instead of projections.** Fails on the thin-rectangle-slicing-through case, where no corner lands inside either shape.
3. **Checking only one axis.** Two rectangles can overlap on `x` but miss entirely on `y` (stacked vertically with no shared row) — both axes are required.
4. **Mixing up which rectangle's coordinate goes on which side of the inequality.** Swapping `x1 < x4` for `x1 < x2` checks a rectangle against itself, not against the other rectangle.
5. **Assuming a naive "does the start of one come before the end of the other" check in one shot without also checking the reverse.** You need both directions (`x1 < x4` and `x3 < x2`), not just one — a single-direction check alone gives false positives, exactly like the 2:00–3:00 vs. 3:15–3:30 calendar example.

Interview Questions

- How would you extend this to return the actual overlapping rectangle, not just a boolean?

- What if the input allowed rectangles with zero area — how does your solution behave, and should it?
- Can you generalize this projection trick to 3D boxes?
- How would you check overlap among N rectangles efficiently, instead of just two?
- Why does checking "when they miss" simplify the logic compared to checking "when they hit" directly?

Similar Problems

- **LeetCode 435, Non-overlapping Intervals** — same 1D interval-overlap logic, applied to interval scheduling and removal.
- **LeetCode 986, Interval List Intersections** — extends the two-interval overlap check into merging many interval lists.
- **LeetCode 252/253, Meeting Rooms I/II** — the calendar-conflict framing from this video's intro, solved directly with interval overlap checks.
- **LeetCode 223, Rectangle Area** — builds on this problem's projection idea to compute actual overlapping area, not just a boolean.

Key Takeaways

Rectangle Overlap teaches a pattern bigger than the problem itself: when a condition is hard to state directly, try stating its complement instead. "When do they miss" decomposed cleanly into two independent 1D interval checks, while "when do they overlap" tempted a geometric, corner-based approach that's actually wrong. Strict inequalities are what separate touching from overlapping, and that single detail is the whole difficulty rating of this "easy" problem.

Watch the Video

For the full walkthrough — including the two quiz rounds that stress-test the strict inequality and the four-comparison count — watch the video here: {LINK}

About the Series

This breakdown is part of **Fun with Learning Technology**, a series built around taking LeetCode's daily challenge and unpacking not just the code, but the reasoning that gets you there — the same instincts that make the difference in a real coding interview.

Call To Action

If this reframe clicked for you, give the video a like on YouTube and subscribe for daily LeetCode breakdowns. Subscribe to the Substack for the written version delivered straight to your inbox. Drop a comment with where else you've used the "check when it misses" trick, and share this with someone prepping for interviews.

Quick Review

Overlapping-rectangles problem: what's the trick?

Don't check overlap directly — check the opposite: when do they NOT overlap on each axis, then negate.

Rectangle Overlap: time complexity?

$O(1)$ — fixed number of coordinate comparisons regardless of input size.

Rectangle Overlap: space complexity?

$O(1)$ — only a few scalar comparisons, no extra structures.

Common bug when writing the overlap condition?

Using $\leq$ instead of $<$ for the separation check — edge-touching rectangles count as non-overlapping, not overlapping.

1D interval overlap vs 2D rectangle overlap?

Rectangle overlap is just two independent 1D interval-overlap checks, one per axis, ANDed together.

Interview follow-up: what if rectangles can touch at an edge?

Strict inequalities already exclude that case — area of overlap must be strictly positive on both axes.